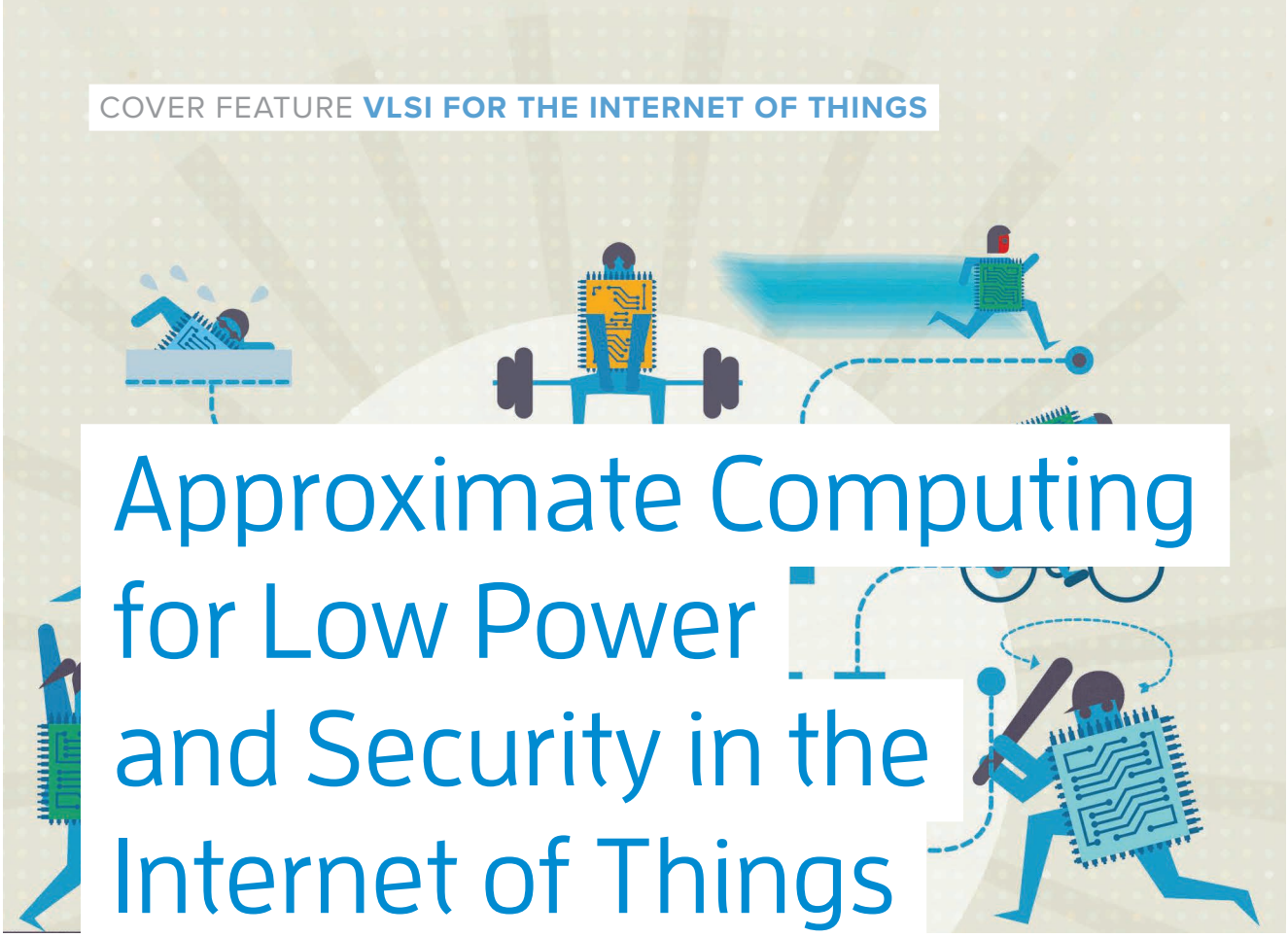


Approximate Computing for Low Power and Security in the Internet of Things

Mingze Gao, Qian Wang, and Md Tanvir Arafin, University of Maryland, College Park
Yongqiang Lyu, Tsinghua University
Gang Qu, University of Maryland, College Park

To save resources for Internet of Things (IoT) devices, a proposed approach segments operands and corresponding basic arithmetic operations that can be carried out by approximate function units for almost all applications. The approach also increases the security of IoT devices by hiding information for IP watermarking, digital fingerprinting, and lightweight encryption.

The Internet of Things (IoT) has become ubiquitous in our daily life, with an estimated 30–75 million IoT devices expected to be connected through the Internet infrastructure by 2020.[1,2] The explosive growth in the number and diversity of these devices and their applications are inevitably bringing many nontraditional challenges to the circuit and system design community.[3] Perhaps the most crucial challenge is how to power these tens of billions of devices, as a significant portion of them are wireless and battery operated, with power consumption directly affecting their lifetime.[3,4] Security and privacy are another concern because IoT devices cannot afford resource-demanding cryptographic protocols. Here, we assess a popular low-power design methodology known as approximate computing and demonstrate how it can both reduce power and increase the security of IoT devices.

APPROXIMATE COMPUTING

Many low-power VLSI design techniques—including dynamic power and thermal management, dynamic voltage scaling, multiple-threshold voltage design, and

 0018-9162/17/$33.00 © 2017 IEEE

TABLE 1. Area and power cost comparison of approximate function units (FUs) of various bit widths.*

FU bit width	Adder			Multiplier		
	No. cells	Area (μm^2)	Power (nW)	No. cells	Area (μm^2)	Power (nW)
8	91	212.12	752,614.84	377	1,037.62	2,829,745.10
16	230	322.33	2,234,652.24	1,406	4,208.68	10,815,807.06
32	498	1,116.93	3,818,821.94	4,916	15,126.00	34,033,690.30

*The shaded area indicates the value normalized to that of precise 32-bit FUs.

energy harvesting—along with the use of nanoscale materials like memristors can improve the power efficiency of IoT devices. To complement these approaches, researchers have proposed novel methods to leverage specific features in IoT applications for power reduction.

One such method is approximate computing, which offers resource savings at the cost of reduced accuracy and has found applications in many fields such as multimedia and signal processing, wireless communication, and data mining and analysis. A common feature of these applications is their ability to tolerate a certain amount of computational error. In this respect, approximate computing is conceptually similar to the use of probabilistic design techniques to reduce energy in multimedia applications, an approach that has been around for more than a decade. As far back as 2003, researchers proposed dropping the decoding of some frames as long as humans cannot tell the difference.[5] The error tolerance in image compression later motivated development of an adaptive pixel and coefficient truncation technique for energy minimization at system level.[6,7]

Generally, low-power approximate computing is achieved in a system either by using voltage overscaling (VOS) techniques to reduce the system's operating voltage below the required threshold level or by redesigning the system's functionality at different levels—algorithm, application, architecture, or circuit—to save power at the cost of computational accuracy or correctness. A recent survey classifies state-of-the-art approximate computing approaches at the levels of computing, software, compilers, architecture, memory, and circuit.[8]

Most existing research focuses on the arithmetic field, where the power/energy savings can be achieved by well-designed approximate computing function units (FUs) such as approximate adders[9] and multipliers.[10] To study the potential of these approximate FUs, we design circuits in Verilog for adders and multipliers with different bit widths and synthesize them using the Cadence RTL Compiler with the FreePDK 45-nm open cell library. As Table 1 shows, the area and power consumption of adders decrease about linearly with the number of bits, while the area and power savings of multipliers are almost exponential. Most approximate FUs take advantage of this by using adders and multipliers with fewer bits whenever possible.

However, existing approximate FUs are fully customized for a specific operation, either addition or multiplication, with a given accuracy. When FUs are used for other computations—for example, approximate adders for multiplication—the error can accumulate and get out of control. More importantly, the large data range significantly limits the power savings of approximate FUs from reaching their potential. Consider a 32-bit adder: when both operands have small values with many leading zeros, an approximate adder with fewer bits could be used to add only the non-zero bits; on the other hand, if one operand or both are large, less significant bits could be neglected without causing significant error, thereby allowing use of an approximate adder. Existing approximate adders with a fixed bit position for addition cannot address both scenarios.

APPROXIMATE ADDITION BASED ON APPROXIMATE INTEGER FORMAT

To save more system resources, we extend approximate computing from the existing static approach to a more data-oriented dynamic one. We achieve this via a segmentation of operands based on approximate integer format (AIF) and corresponding basic arithmetic operations that can be carried out by approximate FUs for almost all applications.[11]

An n-bit positive integer N is segmented into $\lceil n/k \rceil$ blocks with k bits in each block (except the first and leading block if N is not a multiple of k). A *valid block* is a block that contains at least one bit of value 1 or is after some valid blocks. The *precision control* (*pc*) value is the number of valid blocks that will be used in the approximate computation. For the ith block of the given positive number, the *sentinel bits* st[i] are defined as:

$$\mathrm{st}[i] = \begin{cases} 1, & \text{block } i \text{ is a valid block} \\ 0, & \text{block } i \text{ is not a valid block} \end{cases}$$

For example, consider two integers 1,500 and 800, which we can express in binary form as $1{,}500_{10}$ = 0000 **0101 1101 1100**$_2$ and 800_{10} = 0000 **0011 0010 0000**$_2$, respectively. When we choose the block size k = 4, both values will have three valid blocks (shown in

bold) and the same sentinel bits, 0111. If we set $pc = 2$ for an 8-bit approximate FU, the two underlined valid blocks will be used during the approximate computation.

In AIF, data is rounded and stored based on its sentinel bits. Consider a 4-block operand, $A = b_3b_2b_1b_0$, where each b_i is a block of fixed size. A's sentinel bits will have five possible values, st_a: 0000, 0001, 0011, 0111, and 1111. For the first four cases, the value of A will be stored in the following format: $A = st_ab_2b_1b_0$. This will not incur any data loss because the first block, b_3, is a block of all 0s in these cases. In the last case, when $st_a = 1111$, the data A will be stored as $A = st_ab_3b_2b_1$ after we drop the last block, b_0. This will introduce error when the block is not 0, but because the first and most significant block, b_3, is valid, this error will be very small.

Now with all data represented in AIF, we can perform approximate computing in three phases: use sentinel bits to select the blocks for computation; use accurate computing units of smaller bit width to compute with the selected blocks of data; and convert the result to AIF format (for future computation) or report the result. The following algorithm shows how to add two operands, A and B, in AIF, and Figure 1 provides an example.

1. Compute the sentinel bits of the result S: $st_s = st_A \mid st_B$, where | is the bit-wise OR.
2. Supposing the leading bit 1 in st_s is at st[*i*] and *pc* valid blocks will be used, take the bits from the *i*th to the $(i - pc + 1)$th blocks of A and B and name them as A′ and B′, respectively.
3. Compute S′ = A′ + B′ and the possible carry-out, Cout.
4. Update st_s by $st_s[i + 1]$ = Cout.
5. Reconstruct S in AIF based on st_s and S′. Pad with 0s if necessary.

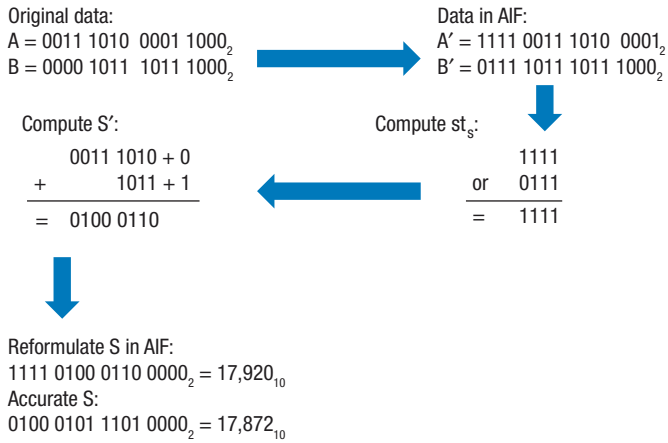


FIGURE 1. Example of approximate addition in approximate integer format (AIF), where $k = 4$, $pc = 2$, and an 8-bit adder is used to compute S′.

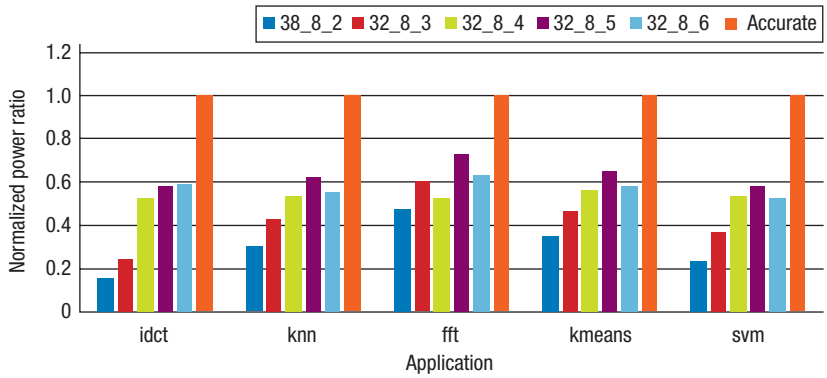


FIGURE 2. Normalized power consumption of different approximate computing configurations for five popular applications. For each configuration: number of operands_number of blocks (with $k = 4$)_precision control value.

Unlike existing approximate arithmetic units, which are designed for specific operations, an AIF-based approach is suitable for all arithmetic operations—addition, subtraction, multiplication, and division—without needing to redesign the arithmetic units. More importantly, AIF can be conveniently extended to fixed-point data and incorporated into high-level programming languages, making it applicable to a large range of real-life applications (see Figure 2) and letting designers control the accuracy–power tradeoff at a high level. Technical details can be found elsewhere.[11]

Besides its effectiveness in saving power, an AIF-based approach guarantees computation quality. Table 2 lists the error of the first 40 terms in the Fibonacci sequence computed by approximation addition with different parameters (data is partitioned into 8 blocks and the *pc* value varies between 2 and 4). From the table, we see that the errors are small, relatively unbiased, and decrease as the *pc* value increases. When $pc = 4$, we correctly generate the first 24 terms with no errors. This is better than any other existing approximation method.

OTHER LOW-POWER VLSI TECHNIQUES FOR THE IOT

In addition to approximate computing, there are many effective techniques

TABLE 2. Fibonacci sequence error of the first 40 terms computed using approximate addition under different configurations.

Term no.	*pc* = 2	*pc* = 3	*pc* = 4
1–13	0	0	0
14	–0.00984	0	0
15	–0.01317	0	0
16	–0.01691	0	0
17	–0.01858	0	0
18	–0.01985	0	0
19	–0.02173	–0.00044	0
20	–0.02905	0.000183	0
21	–0.02625	–5.65E–05	0
22	–0.02732	3.49E–05	0
23	–0.02692	0	0
24	–0.02707	1.33E–05	0
25	–0.02912	0.000404	8.24E–06
26	–0.03225	0.000336	1.02E–05
27	–0.03750	0.000362	9.44E–06
28	–0.03947	0.000352	9.72E–06
29	–0.04118	0.000356	9.61E–06
30	–0.04205	0.000354	9.66E–06
31	–0.02291	–0.000350	2.30E–06
32	–0.02267	–0.000590	–8.51E–06
33	–0.02276	–0.000500	–4.38E–06
34	–0.02273	–0.000530	–5.96E–06
35	–0.02274	–0.000520	–5.36E–06
36	–0.02274	–0.000520	–5.59E–06
37	–0.03280	–0.000100	1.05E–06
38	–0.03621	–0.000460	–5.53E–06
39	–0.04003	–0.000640	–3.02E–06
40	–0.04174	–0.000770	–3.98E–06

to meet the low-power demands of IoT systems.

Circuit-level low-power techniques. At the circuit level, three types of power are dissipated: dynamic power, leakage power, and short-circuit power. In today's VLSI technology, dynamic and leakage power are the dominant sources. Researchers have proposed various circuit-level methods for power and energy minimization. These include clock gating, transistor sizing, glitch and path balancing, technology mapping, temperature/thermal awareness, and dual-threshold voltage and input vector control, and don't-care condition optimization.

Dynamic voltage and frequency scheduling. DVFS varies the supply voltage and clock frequency based on the computational load and deadline requirements to provide the required performance while minimizing the total amount of energy consumption. State-of-the-art DVFS design has been proposed for the functionalities that are conventionally used in IoT devices. This can be naturally integrated with approximate FUs.

Probabilistic design. This design methodology aims at not overdesigning systems for their worst-case scenario.[5] It utilizes the fact that many real-time IoT applications do not require high performance and can tolerate a certain degree of faultiness. Probabilistic design uses prior or posterior execution information and takes advantages of the unique features of IoT devices' functionalities to relax overly rigid hardware or software design implementations.

IOT SECURITY CHALLENGES

Approximate computing can also help address several IoT security challenges. Because IoT devices collect, process, and exchange massive amounts of data that could be confidential or privacy sensitive, they must be robust to potentially malicious attacks and design vulnerabilities, such as the injection of Trojans and other malware into software and hardware, unsophisticated electronic design automation (EDA) tools,[3] and untrusted supply chains. Although there are many well-designed security protocols

based on modern cryptography, they are too computationally expensive and power hungry to be suitable for IoT devices' low-power design requirements. As a result, many IoT devices such as implantable medical devices have no data protection.

Active research on lightweight cryptography aims to deliver affordable but weak security (for example, with short cryptographic keys) to IoT devices[12,13] as well as to hardware security primitives such as silicon physical unclonable functions (PUFs) and hardware-based random-number generators used for authentication. These approaches are much more energy efficient than classic cryptographic solutions. However, they add security as a nonfunctional feature to the system and therefore need hardware or software support.

Hardware-based attacks

As the fundamental element in the IoT, hardware is becoming an attractive new attack surface using various techniques.

Hardware Trojan. This is a piece of circuit implanted in a system design or modified from the original design for malicious purposes. As simple as several logic gates, a hardware Trojan can cause severe damage by, for example, altering or disabling certain FUs, leaking sensitive information, or shortening the lifetime of IoT devices.

Side-channel attack. This classic noninvasive and passive attack monitors, measures, and analyzes a system's physical characteristics leaking from side channels when the system is running. These characteristics include timing, current, voltage, electromagnetic radiation, power consumption, and optical or acoustic information. Side-channel attacks can be very effective against IoT devices, and they are easy to implement and hard to detect and prevent. Because they target vulnerabilities in the hardware or software implementation instead of the algorithms or protocols, theoretically proved secure algorithms or protocols can also become vulnerable.

Reverse engineering. RE is the invasive process of extracting IP from an IoT device and reproducing it based on the obtained information with little or no R&D. These low-cost illegitimate products can be sold at a much lower price, giving them an unfair competitive edge against the authentic products. Moreover, after extracting the IoT device's high-level functionality, the attacker can redesign the device to avoid copyright infringement or insert a hardware Trojan for malicious purposes. RE attacks can be very effective against IoT devices because many are simple and do not use the latest semiconductor technology (which raises the cost of RE tools).

Countermeasures

The hardware design community has responded to these challenges with several countermeasures.

IP watermarking and digital fingerprinting. IP watermarking involves embedding the IP author's signature in a circuit to claim IP rights and is used to detect and prevent IP piracy. A carefully designed watermark can provide high confidence of IP authorship, incurs low design overhead, and is resilient against various watermark removal and modification techniques. However, a watermark remains the same for all copies of a given IP and cannot trace the source of illegally distributed IPs. Digital fingerprinting addresses this problem by embedding the IP user's information together with the IP author's signature. Fingerprinted IPs can thus be viewed as multiple, distinct watermarked IPs. The goal is to identify each copy of the IP and protect honest IP users.

Circuit obfuscation. While digital watermarking and fingerprinting techniques can deter IP piracy, they do not prevent RE. Circuit obfuscation modifies the design and implementation of a circuit to make it difficult to interpret the layout and thus increases the cost and complexity of RE attacks. There are two types of obfuscation techniques. *Logic encryption* inserts additional key gates that have the secret key values as part of their input signals; the key gates and thus the entire circuit will malfunction with incorrect key values. *Circuit camouflage* replaces the original logic gates with configurable logic cells. These cells can be configured to perform as different logic gates, but the difference between configurations is too small to be observed by existing RE tools.

Physical unclonable function. A PUF is a small piece of circuitry embedded in the design that extracts a silicon chip's fabrication variation and uses this intrinsic physical feature for security applications. A PUF can be effective for secret key storage, random number generation, chip authentication, IP protection, and anti-counterfeiting. While a promising hardware security primitive for IoT devices, its usability is limited due to unreliability under different operating environments such as supply voltage, temperature, and humidity as well as circuit aging. Therefore, researchers have proposed

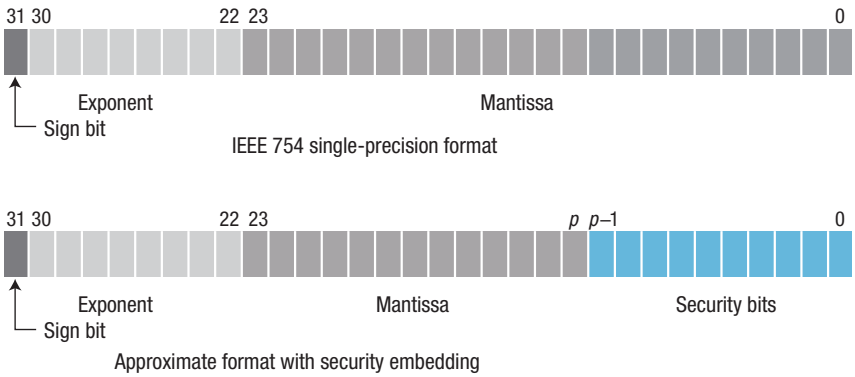


FIGURE 3. IEEE 754 single-precision floating-point format for 32-bit data. The last p bits can be used as security bits to embed information without changing the rest of the $32 - p$ bits.

various means to enhance PUF robustness and stability.

APPROXIMATE ARITHMETIC COMPUTING–BASED INFORMATION HIDING

To minimize the power cost of IoT devices while still providing a practical security solution, we introduce a novel approximate computing–based approach to embed information for authentication and other security-related applications. The idea is inspired by data segmentation, with operands divided into most significant bits (MSBs) and least significant bits (LSBs). In approximate computing, MSBs are used for precise operation and should be preserved, but LSBs can either be ignored or replaced by simple operations such as OR. Our approach hides information into LSBs such that it does not affect approximate computation but can be recovered for security purposes. The implementation requires slight modification to the arithmetic unit—for example, the adder or multiplier—without building extra hardware primitives and corresponding systems.

Floating-point format with security embedding

IEEE 754 specifies an approximate single-precision floating-point format for 32-bit data, which consists of 1 sign bit, 8 exponent bits, and 23 mantissa bits,[14] as the top of Figure 3 shows. (Double precision is very expensive and seldom used in IoT devices.) The value of an IEEE 754 number is computed as sign × mantissa × 2^{exponent}, where the sign is 1 or –1 if the leading bit is 0 or 1, respectively; the mantissa is a real value between 1.0 and 2.0, with a fractional part represented in binary format; and the exponent equals the 8 bits in the middle minus 127. For example, 0,10000000,10010010000111111010000 is 1 × 1.570795 … × 2, which is roughly 3.14159 in decimal format.

Obviously, the LSBs in the mantissa will have little impact on the value. We thus propose using the last p bits as *security bits*, as shown in the bottom of Figure 3, to embed information without changing the rest of the $32 - p$ bits. In the above example, setting the last 10 bits to 0s will make the value 3.1413574, only a 0.0074 percent decrease. The error introduced by any changes to the last p bits will be less than 2^{p-24} relative to the original value.

Approximate computing with information embedding

Given two real numbers A and B, we rewrite them in the approximate data format: $A = A' \oplus K_A$ and $B = B' \oplus K_B$, where A′ and B′ are identical to A and B except that the last p mantissa bits are replaced by 0s; K_A and K_B are the last p bits of A and B; and ⊕ is the bitwise XOR for the last p bits. For any binary arithmetic operation A ⊕ B, we propose the following method to perform approximate computing and information embedding simultaneously:

1. Rewrite A and B in the approximate data format.
2. Compute $A' \oplus B'$ and rewrite it as $O' \oplus K_{O'}$.
3. Generate a p-bit secret key K_S to be embedded.
4. Return $O' \oplus K_S$ as the result of $A \oplus B$.

For example, if A = 3.14159 and B = 12.31, A × B will be 38.6729729. Figure 4 shows the result of following the above scheme with $p = 10$, in which K_S is generated using the simple bitwise XOR of K_A,K_B,$K_{O'}$ and a random key = 01010101. The output value is 38.67124, only 0.00448 percent less than the accurate result of 38.6729729.

Information hiding for security applications

K_S can be as simple as a constant or some function of K_A, K_B, $K_{O'}$ and Key.

In general, we can write this as $K_S = F(K_A, K_B, K_{O'}, \text{Key})$. K_S can be generated and used for IP watermarking, digital fingerprinting, and lightweight encryption.

IP watermarking. During IP design and implementation, either the IP owner's digital signature can be used as the Key or operand values can be selected to enable the above proposed steps—that is, a mechanism such as returning $A' \oplus B'$ directly (and skipping steps 3 and 4) unless the operands match the given values when the watermark is embedded or a watermark verification signal is activated. Checking the output error then reveals the watermark.

Digital fingerprinting. Similar to IP watermarking, each device's unique fingerprint can be embedded as the error value in the LSBs. For the same verification inputs or operands, different Key values and F()s can be used to distinguish individual devices.

Lightweight encryption. As the result will be encrypted, the entire 32-bit space can be used for information hiding. For example, an encryption key up to 32 bits can be generated from F() based on the values of operands and some given Key values to encrypt the (approximated) computational result—for example, by the efficient bitwise XOR operation. This is symmetric encryption and the result can be decrypted easily once the key is available.

There are two potential threats to this information-hiding protocol. First, an attacker can study the values of the operands and results to reveal the Key value, the secret K_S, and the function that generates K_S. Second, authorized parties will reveal the hidden information to verify the watermark and fingerprint or decrypt the encrypted results. An insider attacker or someone who has managed to gain this permission can further alter or forge the hidden information. Such attacks can be easily prevented by enhancing the proposed method with additional cost. For example, the simple XOR operation can be replaced by a one-way hash function, information can be hidden in multiple operands and at different computational stages during a complex operation, and intrinsic hardware features such PUFs and secure memory can be utilized to secure Key and K_S.

The main advantage of our proposed approximate computing–based information-hiding method is its low implementation cost with guaranteed results: it utilizes the energy efficiency of approximate computing and introduces tolerable error. The hidden information can be verified easily by disabling the approximate computing. This security primitive can provide lightweight security for multiple security applications such as device authentication and IP protection. ■

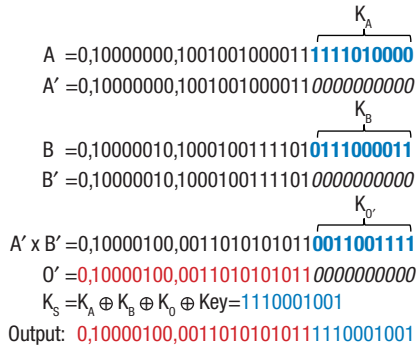


FIGURE 4. Example of approximate computing with information hiding. If $p = 10$, a secret key K_S is generated using the simple bitwise XOR of K_A,K_B,$K_{O'}$ and a random key = 01010101. The output value is 38.67124, only 0.00448 percent less than the accurate result of 38.6729729.

ACKNOWLEDGMENTS

This work was supported in part by AFOSR MURI under award number FA9550-14-1-0351. Yongqiang Lyu was supported in part by Tsinghua University Initiative Scientific Research Program and Chinese National Key Research and Development Program 2016YFB1000102.

REFERENCES

1. "More Than 30 Billion Devices Will Wirelessly Connect to the Internet of Everything in 2020," press release, ABI Research, 9 May 2013; www.abiresearch.com/press/more-than-30-billion-devices-will-wirelessly-conne.
2. T. Danova, "Morgan Stanley: 75 Billion Devices Will Be Connected to the Internet of Things by 2020," *Business Insider*, 2 Oct. 2013; www.businessinsider.com/75-billion-devices-will-be-connected-to-the-internet-by-2020-2013-10.
3. G. Qu and L. Yuan, "Design Things for the Internet of Things: An EDA Perspective," *Proc. IEEE/ACM Int'l Conf. Computer-Aided Design* (ICCAD 14), 2014, pp. 411–416.
4. H. Jayakumar et al., "Powering the Internet of Things," *Proc. Int'l Symp. Low Power Electronics and Design* (ISLPED 14), 2014, pp. 375–380.
5. S. Hua, G. Qu, and S.S. Bhattacharyya, "Energy Reduction Techniques for Multimedia Applications with Tolerance to Deadline Misses," *Proc. 40th Ann. Design Automation Conf.* (DAC 03), 2003, pp. 131–136.
6. S.H. Kim, S. Mukhopadhyay, and M. Wolf, "System-Level Energy Optimization for Error-Tolerant Image Compression," *IEEE Embedded System Letters*, vol. 2, no. 3, 2010, pp. 81–84.
7. S.H. Kim, S. Mukhopadhyay, and M. Wolf, "Modeling and Analysis of Image Dependence and Its Implications for Energy Savings in Error Tolerant Image Processing," *IEEE Trans.*

Computer-Aided Design of Integrated Circuits and Systems, vol. 30, no. 8, 2011, pp. 1163–1172.
8. Q. Xu, T. Mytkowicz, and N.S. Kim, "Approximate Computing: A Survey," *IEEE Design & Test*, vol. 33, no. 1, 2016, pp. 8–22.
9. R. Ye et al., "On Reconfiguration-Oriented Approximate Adder Design and Its Application," *Proc. Int'l Conf. Computer-Aided Design* (ICCAD 13), 2013, pp. 48–54.
10. S. Hashemi, R.I. Bahar, and S. Reda, "DRUM: A Dynamic Range Unbiased Multiplier for Approximate Applications," *Proc. IEEE/ACM Int'l Conf. Computer-Aided Design* (ICCAD 15), 2015, pp. 418–425.
11. M. Gao et al., "A Novel Data Format for Approximate Arithmetic Computing," *Proc. 22nd Asia and South Pacific Design Automation Conf.* (ASP-DAC 17), 2017; doi:10.1109/ASPDAC.2017.7858354.
12. M. Katagi and S. Moriai, "Lightweight Cryptography for the Internet of Things," Sony Corp., 2008; www.iab.org/wp-content/IAB-uploads/2011/03/Kaftan.pdf.
13. M.T. Arafin, M. Gao, and G. Qu, "VOLtA: Voltage Over-Scaling Based Lightweight Authentication for IoT Applications," *Proc. 22nd Asia and South Pacific Design Automation Conf.* (ASP-DAC 17), 2017; doi:10.1109/ASPDAC.2017.7858345.
14. *IEEE Std. 754-2008, IEEE Standard for Floating-Point Arithmetic*, IEEE, 2008.

ABOUT THE AUTHORS

MINGZE GAO is a PhD student in the Department of Electrical and Computer Engineering at the University of Maryland, College Park. His research interests include hardware security, low-power design, and approximate computing. Contact him at mgao1@umd.edu.

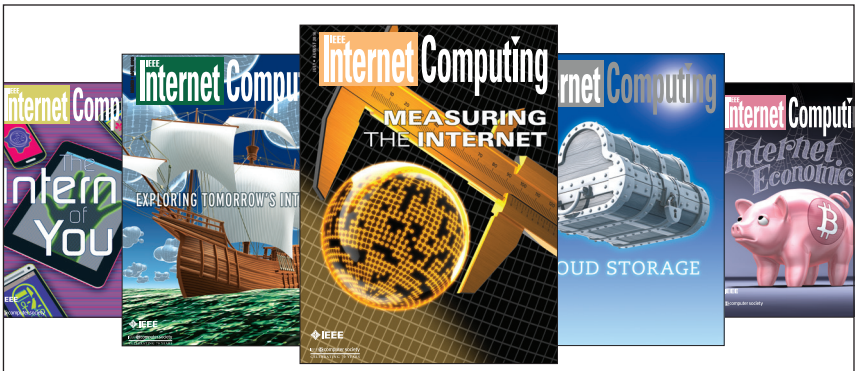
QIAN WANG is a PhD student in the Maryland Embedded Systems and Hardware Security (MeshSec) lab at the University of Maryland, College Park. Her research interests include embedded systems and hardware security. Contact her at qwang126@umd.edu.

MD TANVIR ARAFIN is a PhD student in the Department of Electrical and Computer Engineering at the University of Maryland, College Park. His research interests include semiconductor physics, integrated circuits, and embedded security of microelectronic devices. Contact him at marafin@umd.edu.

YONGQIANG LYU is an associate professor at the Research Institute of Information Technology at Tsinghua University. His research focuses on the hardware–software fusion architecture in advanced computing systems and high-performance networks. Lyu received a PhD in computer science from Tsinghua University. He is a member of IEEE, ACM, and the China Computer Federation. Contact him at luyq@tsinghua.edu.cn.

GANG QU is a professor in the Department of Electrical and Computer Engineering and at the Institute for Systems Research at the University of Maryland, College Park, where he also directs the MeshSec and Wireless Sensor labs. His research interests include optimization and combinatorial problems and their application to very large-scale integration and computer-aided design, wireless sensor networks, bioinformatics, cybersecurity, and hardware security and its impact on Internet of Things and embedded systems. Qu received a PhD in computer science from the University of California, Los Angeles. He is a Senior Member of IEEE. Contact him at gangqu@umd.edu.

myCS Read your subscriptions through the myCS publications portal at **http://mycs.computer.org**



Want to know more about the Internet?
This magazine covers all aspects of Internet computing, from programming and standards to security and networking.
www.computer.org/internet